i4r-audit — team cheat sheet

Set up once, then: reserve a paper → run the audit → file the result. Every dark window below is a **verbatim capture from a real run**, not an illustration. Your audit's exact gate set is stamped into its `run.json` as `gates-<hash>`.

1 What you need (one-time, ~30 minutes)

Thing	Where / how	Why
Claude Code, on an account that can run Sonnet 5	`npm install -g @anthropic-ai/claude-code`, then run `claude` and log in	The coordinator and the five auditors run here
ChatGPT desktop app, logged in, workspace with credits	chatgpt.com/download — the Codex CLI ships inside the app; nothing extra to install	Ordinary adversarial critics run on GPT 5.6 Terra/high; targeted adjudication uses Terra/max. **No credits = no critique rounds** — the skill detects this and stops before wasting one
R ≥ 4.4 with the notebook packages	`install.packages(c("haven", "lavaan", "psych", "rmarkdown", "knitr", "readxl", "jsonlite"))`	The verification notebook re-derives every number in the comment
Python 3 with the readers	`pip3 install pyreadstat pandas scipy numpy statsmodels jsonschema openpyxl`	Auditors cross-read every data format two ways
pandoc + LibreOffice	`brew install pandoc` · libreoffice.org	Builds the .docx and measures the real page count against the 10-page cap
The shared Dropbox discipline folder, synced locally	Ask the coordinator for the invite	Papers, packages, reservations, and completed reports live in one tree
The skill itself	Next section	

2 Install and prove it works

Terminal — install + self-tests

```
$ unzip i4r-audit.zip && cp -R i4r-audit ~/.claude/skills/
$ chmod +x ~/.claude/skills/i4r-audit/scripts/*.sh
$ bash ~/.claude/skills/i4r-audit/scripts/make_claims_script.sh --selftest
SELFTEST OK: 12 checks — R fallback, Stata-primary naming, author-inline, R-embedded, severity, tail
$ bash ~/.claude/skills/i4r-audit/scripts/reserve_paper.sh --selftest
SELFTEST OK: 8 checks — claim, contention, idempotence, foreign release, conflict copy, completed guard, stale flag
```

If either prints `FAIL`, stop and ask in the team channel before running anything.

The three iron rules

- **Never contact the paper's authors or the journal.** All correspondence goes through I4R — no exceptions, not even a "quick clarifying question."
- **Cannot verify a RETRACTION- or CORRIGENDUM-level claim after honest study? Return the paper** and take another. Never skip it; never let the AI decide for you.
- **Never name the discipline** in anything a replicator will see — every document is reused verbatim for the next cohort.

3 Reservations — automatic, but know what they look like

Six people work this tree at once. **The skill claims the paper for you the moment an audit starts** and aborts cleanly if someone else holds it — you never reserve by hand before running. The commands below are for checking the board, claiming a paper ahead of time, or giving one back.

```
Terminal — claim a paper
$ export I4R_OWNER=yourname@yourmachine   # once per shell; defaults to user@host

# what the AUTOMATIC claim looks like when the audit starts (you do nothing):
  [reserve] claim written; waiting 15s for sync conflicts ...
  [reserve] RESERVED: Example Paper Title (2024)  ->
.../reservations/Journal_of_Example_Studies_Example_Paper_Title_2024_.json

# and when someone else already holds the paper — the audit stops before doing any work:
  [reserve] REFUSED: reserved by ana@macbook since 2026-08-24T20:06:32Z
  FATAL: could not reserve this paper — someone else holds it or a conflict is unresolved.

$ bash ~/.claude/skills/i4r-audit/scripts/reserve_paper.sh list "<discipline-root>"
         ana@macbook                  0d  Journal_of_Example_Studies_Example_Paper_Title_2024_
  STALE  carol@lab-imac              9d  Another_Journal_Old_Claim_
```

The 15-second wait lets Dropbox surface a same-moment competing claim as a "conflicted copy" — the **earlier timestamp wins automatically**. A 7-day-old claim with no completed report shows `STALE`; release it with `release <paper-dir> --force` only after checking with its owner. Your reservation is claimed at start and released at filing — both automatic.

4 Run the audit

Open a terminal **in the paper's folder** and start Claude Code. Two settings before you paste anything: type `/model`, pick **Sonnet 5**, and set the reasoning effort to **max** — the audit's sub-agents carry their own settings, but the coordinator's ceiling is whatever you select here, and a missed finding costs more than the extra thinking. Then prompt through **`/goal`** rather than a plain message: a goal keeps Claude working until its stated condition is met, restarting after every pause instead of stopping to report progress. Phrase the goal on the **deliverable**, not the input path — long sessions compact their history, and "the audit is filed" stays checkable when a path reference no longer is.

PASTE INTO CLAUDE CODE — AFTER /MODEL → SONNET 5, EFFORT MAX

```
/goal Audit the paper in this folder end to end. The article PDF is ./paper.pdf and the replication
package is ./package.zip. Run the whole pipeline without stopping for approval: workspace, five
auditors, findings-registry projection and adjudication, critique rounds to the production default,
verification notebook, deliverable, claims script. Done means: finalize_run.sh reports the registry
pair filed, and publish_completed.sh has copied the six-file release into Completed reports.
```

setup	five auditors	registry + adjudication	critique rd 1	critique rd 2	build + file
~5 min	40–60 min	1–2 h	45–60 min	~40 min	20–40 min
environment recorded	parallel, one question each	every finding dispositioned	4 critics attack the audit	2 critics, scoped	notebook, gates, publish

historical baseline 4–6 hours; the v2 optimization target is 2.6–3.9 hours, subject to benchmark gates

You mostly watch. Two moments need a human: the coordinator may surface an **adjudication it re-ran** (read it — that is the audit's core), and any **gate FAIL** stops the pipeline with an explanation.

5 While it runs: how to check on it

Watch without interrupting. Everything narrates into files as it happens; a second terminal shows where the run is:

Overnight: run `caffeinate -i` in another tab — a sleeping Mac pauses everything mid-round (harmless; it resumes on

```
Terminal — second window, read-only
$ ls audit/report/            # auditor reports appear one by one
$ python3 -m json.tool audit/critique/round_1/ROUND_STATUS.json
  "outcome": "running"
  "detail": "model probe accepted; critics running"
```

wake). If you must take the machine, close the terminal: re-running the same round later **resumes** — completed critics are skipped, nothing repeats.

Interject freely. A `/goal` session answers questions mid-run ("what did auditor 2 find?") without derailing the pipeline.

Read the adjudications. When the coordinator re-runs a disputed computation it says so — that is the audit's core, and it deserves your eyes more than any other output.

6 What lands where

```
<Paper folder>/
  audit/ — full working record: reports, ledger, adjudication, critique rounds, notebook, run.json
  I4R_comment_<Author>_<Year>.pdf
<discipline-root>/
  Completed reports/<Journal>/<Paper>/
    exactly 6 files: PDF · claims script · comment.json ·
    run.json · finalization receipt · release manifest
  reservations/ — your claim vanishes on filing
```

```
Terminal — a clean finalize
ok: run.json validates against run.schema.json
ok: comment.json validates against comment.schema.json
ok: the notebook knitted with every claim reproducing
ok: filed into .../registry/<audit-id>
     (index now holds N audits)
```

7 When something goes wrong

You see	It means / do this
`FATAL: the Codex workspace is OUT OF CREDITS`	The probe caught an empty balance **before** burning a round. Workspace owner refills; re-run the same round command.
`FATAL: the critic model '…' is rejected`	The ChatGPT app updated and changed its default model (three versions in ten days). Nothing to fix locally — the skill pins the model. Report it in the channel.
`[fail] critic_x attempt N: exit 1, no output file`	The real error is at the bottom of `critique/round_N/log_*_try*.txt` — usually credits or network.
`REFUSED: reserved by …`	Someone holds the paper. Pick another; check `reserve_paper.sh list`.
`…conflicted copy… in a claim result`	Two claims raced through Dropbox. The output already names the winner (earliest timestamp); the loser picks another paper.
`FAIL: abstract is N words / findings table in body / table-vs-tags mismatch`	The comment drifted from the format (a comment is a paper: Abstract ≤250 words on page 1, findings table in Appendix A). The skill fixes and rebuilds; if you edited by hand, undo and let it.
`WARNING: no PDF page count`	LibreOffice is missing, so the 10-page body cap is unverified. Install it (item 5) and rebuild.
`R notebook exits nonzero`	**A claim genuinely failed to reproduce — that is a finding, not a bug.** Read `verification_summary.csv`. Never "fix" the check to pass.

Historical planning baseline: roughly 2–3M Claude-side tokens and 4–6 hours wall-clock. The optimized target is at least 30% fewer tokens and 35% less wall time, subject to paired benchmark gates. The machine is busy; you are not. `run.json` records your software versions and per-phase costs automatically — you never fill a form.

8 FAQ

Question	Answer
Which paper do I pick?	Any paper in the journal folders with a replication package that `reserve_paper.sh list` shows as free and that is not in Completed reports. The reservation claim at startup enforces both.
Can I run two audits at once?	Yes — separate terminals, separate paper folders. The binding constraints are Codex credits and your machine's memory; two is comfortable, three is pushing it.
My laptop slept / crashed mid-run. Is the audit ruined?	No. Everything on disk survives. Reopen Claude Code in the same folder and re-state the goal; completed auditors and critics are detected and skipped, and a half-finished critique round resumes when re-invoked with the same round number.
I need to abandon a paper halfway. What do I do?	Easiest: just tell Claude “release my reservation on this paper” in the session. Or run `reserve_paper.sh release "<paper-dir>"` yourself. Either way a colleague can take it immediately; if you forget, the claim shows STALE after 7 days and can be force-released — but releasing beats being nagged.
The package has no code at all. Still auditable?	Yes — that is itself a finding. The auditors work from the data and the printed statistics, the checks are written in R, and the claims script's no-code tail carries one manual check per claim.
The paper used SPSS / Stata, which I don't own.	Fine for producing the audit: authors' code is carried verbatim into the claims file (in its own language) and the audit's own checks are R. Only the external verifier may need the original tool, and only for author-code blocks.
How do I know it's actually done?	Three signs, all required: finalize prints its **ok:** lines ending in “filed into …/registry/…”; the paper's folder appears in Completed reports with exactly six files; your reservation is gone from `reservations/`.
Do I read the comment before it counts?	You can, and a skim of the retraction/corrigendum findings never hurts — but this is a pilot: every report will be assigned to a replicator who verifies each claim manually, and the coordinator reads every comment before anything is submitted. Your job is to produce the report cleanly, not to referee it.
Can I edit the comment by hand?	**No — do not edit the reports.** The pipeline's gates, records, and claim numbering all assume the text the machinery produced; a hand edit breaks the joins and skips the checks. If something looks wrong, tell Claude what you saw and let the pipeline regenerate, or flag it in the team channel.
The critics found problems with the audit itself. Is that bad?	That is the design working. Across the calibration audits, most blocking criticisms were defects in our own draft, not the paper — caught before publication instead of after. An audit whose critique rounds find nothing is the suspicious one.
Codex asks me to log in again / says quota exceeded.	Open the ChatGPT app, log in, confirm the workspace has credits, then re-run the round. The skill's probe stops before burning retries, so nothing was lost.
Who pays for the tokens?	The institute's accounts on both sides. Costs are metered per phase into `run.json` automatically — which is also how we will report cost per audit in the paper, so don't disable it.